

Cloud Computing & DevOps

Séries de TD/TP — Master Sciences et Techniques

14 séries de 2 h | Volume total : 28 h de travaux dirigés et pratiques

Version étudiant

Faculté des Sciences et Techniques de Settat — Université Hassan I

Pr. A. Abassi

Présentation

Chaque série associe une courte partie **TD** (raisonnement, conception, analyse, 30 à 40 min) et une partie **TP** (manipulation guidée, 70 à 80 min) afin que chaque notion soit d'abord comprise puis mise en œuvre. Les séries suivent le fil conducteur du module : *Git* → *CI/CD* → *Docker* → *Docker Compose* → *Kubernetes* → *Monitoring*, et préparent directement le mini-projet final.

N°	Titre de la série	Chapitre du cours
1	Caractéristiques du Cloud et étude de cas de migration	Ch. 1 — Introduction au Cloud
2	Classer, concevoir : IaaS, PaaS, SaaS, FaaS	Ch. 2 — Modèles de services
3	Première machine virtuelle Linux et réseau	Ch. 3 — Virtualisation
4	Plusieurs VM : simulation d'une infrastructure Cloud locale	Ch. 3 — Virtualisation
5	Docker : premiers conteneurs et images	Ch. 4 — Docker
6	Dockerfile, volumes et réseaux : conteneuriser une API FastAPI	Ch. 4 — Docker
7	Docker Compose : API FastAPI + PostgreSQL	Ch. 5 — Docker Compose
8	Application complète : Frontend React + API + Base de données	Ch. 5 — Docker Compose
9	Culture DevOps et automatisation locale (Makefile, tests)	Ch. 6 — Introduction au DevOps
10	Git : travail collaboratif, branches et conflits	Ch. 7 — Git
11	Pipeline CI/CD : tests, build d'image, déploiement	Ch. 8 — CI/CD
12	Kubernetes (1) : Minikube, Pod, Deployment, Service	Ch. 9 — Kubernetes
13	Kubernetes (2) : ConfigMap, Secret, persistance, scaling	Ch. 9 — Kubernetes
14	Monitoring, logs et sécurité : Prometheus, Grafana	Ch. 10 — Monitoring, sécurité

Consignes générales et Organisation GitHub

Afin de vous familiariser avec le suivi de version et l'ingénierie logicielle collaborative (DevOps), l'ensemble des travaux du module sera centralisé sur l'organisation GitHub officielle du Master.

Procédure de configuration initiale (Binôme)

1. **Invitation à l'organisation** : Vous recevrez une invitation par e-mail via l'organisation GitHub du cours. Acceptez l'invitation pour accéder à l'espace de travail.
2. **Création du dépôt unique (Monorepo)** : Un seul membre du binôme crée le dépôt distant privé dans l'organisation sous le nom strictement formaté :

`tp-cloud-devops-nom1-nom2`

Exemple : `tp-cloud-devops-alami-benjelloun`.

3. **Membres et permissions** : Ajoutez votre binôme comme collaborateur avec les droits d'écriture (*Write*). L'équipe enseignante doit disposer des accès d'administration/lecture.

Structure attendue du dépôt

Chaque série fera l'objet d'un répertoire dédié à la racine du dépôt. La structure doit respecter scrupuleusement l'arborescence suivante :

```
tp-cloud-devops-nom1-nom2/
  README.md           # Presentation du binome et instructions globales
  .gitignore           # Fichiers a ignorer (binaires, env, logs, etc.)
  serie-01/            # Livrables de la Serie 1
    sim_elasticite.py
    CR_Serie_01.md     # Compte-rendu et captures d'ecran commentees
  serie-02/            # Livrables de la Serie 2
  ...
  ...
```

Règles d'évaluation et de soumission

- **Identité** : Le fichier `README.md` à la racine doit impérativement préciser : noms, prénoms, CNE/CNEs et groupe du binôme.
- **Commits et traçabilité** : Les contributions des deux membres du binôme doivent apparaître dans l'historique des commits Git. Les messages de commit doivent être clairs et explicites (ex : `feat(serie-01): ajout du script de simulation`).
- **Date de rendu** : Le dépôt sur GitHub fait foi pour l'horodatage des livrables. Les modifications apportées après la date limite attribuée à chaque série ne seront pas prises en compte.
- **Environnement de travail** : Les commandes précédées du symbole `$` sont à exécuter dans un terminal Linux (Ubuntu / WSL) ou PowerShell sous Windows (`$` est le symbole d'invitation de commande).
- **Mode hors-ligne** : En cas de restriction réseau en salle de TP, utilisez les images et dépendances locales fournies dans l'Annexe A.

Série 1 — Caractéristiques du Cloud et étude de cas de migration

Chapitre associé	Ch. 1 — Introduction au Cloud Computing
Durée	2 h : TD 45 min, TP 65 min, bilan 10 min
Objectifs	Identifier les caractéristiques d'une architecture Cloud ; distinguer scalabilité, élasticité et disponibilité ; choisir un modèle de déploiement ; quantifier l'intérêt de l'élasticité.
Prérequis	Notions de base du cours (définition NIST, modèles de déploiement), Python 3.

Partie A — TD (45 min)

Exercice 1 — Les caractéristiques essentielles du Cloud

Pour chaque situation, indiquer la caractéristique du Cloud illustrée (libre-service à la demande, accès réseau étendu, mutualisation des ressources, élasticité rapide, service mesuré) :

- Un étudiant crée une machine virtuelle en 2 minutes depuis une console web, sans contacter le fournisseur.
- La facture du mois indique 312 heures de calcul et 40 Go de stockage consommés.
- Le service est accessible depuis un téléphone, une tablette ou un PC via HTTPS.
- Le nombre de serveurs passe de 2 à 10 pendant les inscriptions puis redescend automatiquement.
- Plusieurs clients différents partagent le même parc physique sans voir les données des autres.

Exercice 2 — Scalabilité, élasticité, disponibilité

Associer chaque énoncé au concept le plus adapté et justifier :

- « Nous pouvons passer de 1 000 à 100 000 utilisateurs en ajoutant des machines. »
- « Les ressources s'ajustent automatiquement à la charge, en montée comme en descente. »
- « Le service répond 99,95 % du temps, même en cas de panne d'un serveur. »
- Expliquer la différence entre *scalabilité verticale* (scale up) et *horizontale* (scale out) avec un exemple.

Exercice 3 — Choisir un modèle de déploiement

Pour chaque organisation, proposer un modèle (public, privé, hybride, communautaire) et justifier en deux lignes :

- Une startup de 3 personnes qui lance une application mobile.
- Une faculté : données des étudiants sensibles, mais fortes pointes de charge lors des inscriptions.
- Une banque soumise à une réglementation stricte sur la localisation des données.
- Un consortium d'universités marocaines qui mutualise une plateforme de calcul pour la recherche.

Exercice 4 — Étude de cas : migration de « ScolaNet »

L'application ScolaNet gère la scolarité de 2 000 étudiants. Elle tourne sur un serveur physique unique (8 cœurs, 32 Go de RAM) avec une base PostgreSQL sur la même machine. Le trafic est multiplié par 10 en septembre. L'an dernier, une panne matérielle a provoqué 2 jours d'indisponibilité. Le budget d'investissement est limité.

- Lister les problèmes de l'architecture actuelle.
- Proposer une stratégie de migration (rehost, replatform, refactor) et la justifier.
- Identifier trois risques de la migration et une mesure d'atténuation pour chacun.

4. Proposer un plan de migration en quatre phases.

Partie B — TP (65 min) : mesurer l'élasticité

Étape 1 — Inventaire des ressources de la machine

Relever les ressources de votre poste, qui joue ici le rôle d'un « serveur physique » :

```
# Sous Linux / WSL :
$ lscpu | grep -E "^CPU(s\)|Model name"
$ free -h
$ df -h /
$ ip -br a
```

Remarque

Pour les étudiants sous Windows (PowerShell) :

```
PS> Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores,
NumberOfLogicalProcessors
PS> Get-CimInstance Win32_OperatingSystem | Select-Object TotalVisibleMemorySize,
FreePhysicalMemory
PS> Get-PSDrive C
PS> Get-NetIPAddress -AddressFamily IPv4
```

Si WSL (Windows Subsystem for Linux) ou Git Bash est installé, vous pouvez également exécuter directement les commandes Linux.

Noter : nombre de cœurs, RAM totale, espace disque, adresses IP. Ces informations seront utiles en série 3 pour dimensionner les machines virtuelles.

Étape 2 — Simulation : infrastructure statique ou élastique

Créer le fichier `sim_elasticite.py`. La charge (requêtes/s) est relevée heure par heure sur une journée ; un serveur supporte 100 requêtes/s et coûte 0,5 euro par heure.

```
import math

charge = [120, 80, 60, 50, 50, 70, 150, 300, 520, 640, 700, 680,
600, 620, 650, 700, 560, 400, 300, 250, 200, 180, 150, 130]
CAP = 100      # requetes/s supportees par un serveur
PRIX = 0.5     # euros par serveur-heure

statique = math.ceil(max(charge) / CAP)      # dimensionne pour le pic
h_statique = statique * len(charge)
h_elastique = sum(math.ceil(c / CAP) for c in charge)

print("Serveurs (statique)      :", statique)
print("Serveur-heures statique  :", h_statique, "->", h_statique * PRIX, "EUR")
print("Serveur-heures elastique :", h_elastique, "->", h_elastique * PRIX, "EUR")
print("Economie : %.1f %" % (100 * (1 - h_elastique / h_statique)))
```

Exécuter le script dans un terminal (Linux, macOS, CMD ou PowerShell) :

```
$ python sim_elasticite.py
# Ou selon le système :
$ python3 sim_elasticite.py
```

Étape 3 — Analyse et variantes

1. Relever les résultats et calculer l'économie réalisée par l'élasticité.

2. Ajouter une marge de sécurité de 20 % (remplacer c par $c * 1.2$ dans les deux calculs). Que devient l'économie ?
3. Quelle limite du modèle avez-vous identifiée (temps de démarrage d'un serveur, charge imprévue) ?

Livrable : fichier `sim_elasticite.py`, résultats chiffrés et réponse (5 lignes) à la question : « Pour quel type de charge le Cloud est-il le plus rentable ? »

Annexe A

Installation et exécution de Python (Windows & Linux)

Python est un langage interprété : les programmes ne nécessitent pas de compilation explicite avant exécution, l'interpréteur exécute directement le code source.

1. Installation sous Windows

1. Télécharger l'installateur officiel Windows depuis <https://www.python.org/downloads/>.
2. Lancer l'installateur `.exe`.
3. **Étape cruciale** : Cochez impérativement la case « **Add python.exe to PATH** » (ou *Add Python 3.x to PATH*) en bas de la première fenêtre.
4. Clic sur **Install Now**.
5. Vérifier l'installation : Ouvrir l'application **Invite de commandes (CMD)** ou **PowerShell** et saisir :

```
PS> python --version
```

2. Installation sous Linux (Ubuntu / Debian)

Python 3 est généralement préinstallé. Si nécessaire, exécuter dans le terminal :

```
$ sudo apt update
$ sudo apt install python3 python3-pip -y
$ python3 --version
```

3. Création et exécution du programme

1. Ouvrir un éditeur de texte (VS Code, Notepad++, Bloc-notes, Nano, Vim...).
2. Copier le code Python fourni dans la Série 1.
3. Enregistrer le fichier sous le nom `sim_elasticite.py` (vérifier que l'extension est bien `.py` et non `.py.txt`).
4. Ouvrir un terminal dans le dossier contenant le fichier et lancer l'exécution :

```
# Sous Windows (CMD / PowerShell) :
PS> python sim_elasticite.py

# Sous Linux / macOS :
$ python3 sim_elasticite.py
```

4. Alternative pour compiler en fichier exécutable autonome (.exe)

Si vous souhaitez transformer le programme Python en fichier binaire/exécutable (sans avoir besoin d'installer Python sur d'autres machines) :

```
# Installer l'outil PyInstaller
$ python -m pip install pyinstaller

# Generer l'exécutable autonome
$ pyinstaller --onefile sim_elasticite.py
```

L'exécutable généré se trouvera dans le dossier `dist/` (`sim_elasticite.exe` sous Windows ou `sim_elasticite` sous Linux).

Annexe B

Répartition horaire indicative

Série	Thème	TD	TP
1	Caractéristiques du Cloud, migration	45 min	65 min